

Einführung in die Programmierung mit Python

HSLU Hochschule
Luzern

Syllabus

1. Python installieren; Jupyter Notebooks
2. **Grundlagen: Arithmetik, Variablen und Datentypen**
3. Methoden; Listen; Bedingte Anweisungen
4. Schleifen
5. Benutzerdefinierte Funktionen
6. Datenaufbereitung und Grafische Darstellungen

Python-Grundlagen

Zahlen

Mathematik mit Ganzzahlen funktioniert wie erwartet:

```
In [ ]: 5 + 2
```

```
In [ ]: 2 - 4
```

```
In [ ]: 7 * (6 + 1) # Klammern funktionieren wie üblich
```

```
In [ ]: 2 ** 3 # zwei hoch drei
```

Alles in einer Zelle:

```
In [ ]: 5 + 2  
        2 - 4  
        7 * (6 + 1)  
        2 ** 3
```

Was ist passiert? Jupyter gibt nur das Ergebnis des *letzten* Ausdrucks in einer Zelle aus. Wir können das mit der `print`-Funktion korrigieren:

```
In [ ]: print(5 + 2)  
        print(2 - 4)  
        print(7 * 7)  
        print(2 ** 3)
```

Was ist mit Division?

```
In [ ]: 2 / 3
```

Das funktioniert auch, gibt aber eine andere Art von Zahl zurück: eine Gleitkommazahl oder `float`. Dies gilt auch, wenn die Division prinzipiell exakt durchgeführt werden könnte:

```
In [ ]: 6 / 2
```

Für einen Menschen repräsentieren 3.0 (ein `float`) und 3 (eine Ganzzahl oder `int`) dieselbe Zahl, sie werden jedoch im Speicher unterschiedlich dargestellt; wir sagen, dass diese beiden Objekte einen unterschiedlichen **Typ** haben. Wir können den Typ eines Objekts wie folgt herausfinden:

```
In [ ]: type(3)
```

```
In [ ]: type(3.0)
```

Mathematik mit Gleitkommazahlen kann knifflig sein, da sie mit begrenzter Genauigkeit dargestellt werden, was bedeutet, dass nicht alle Zahlen darstellbar sind:

In []: `1 - 0.9`

Variablen

Eine Variable ist ein benannter Speicherort. Sie wird mit "=" zugewiesen.

```
In [ ]: a = 2  
        b = 4  
        c = a + b  
        print(c)
```

Können Sie erraten, was der folgende Code tut?

```
In [ ]: a = 2  
        a = a + 1  
        print(a)
```

Der folgende Code ist äquivalent:

```
In [ ]: a = 2  
        a += 1 # Kurzform für a = a + 1  
        print(a)
```


Variablennamen können aus Buchstaben, Zahlen und dem Unterstrich bestehen. Sie dürfen nicht mit einer Zahl beginnen. Python ist case-sensitive: `A` ist nicht dasselbe wie `a`.

Zuweisung versus Gleichheit

Wir haben gerade gesehen, dass Variablen mit `=` zugewiesen werden.

```
In [ ]: a = 3  
        print(a)
```

Was, wenn wir prüfen wollen, ob zwei Zahlen gleich sind? Erster Versuch:

```
In [ ]: # Entkommentieren Sie die nächste Zeile und führen Sie die Zelle aus  
        # 3 = 3
```

Das hat offensichtlich nicht funktioniert. Der richtige Weg ist die Verwendung von `==`:

```
In [ ]: 3 == 3
```

```
In [ ]: 1 == 2
```

Das zurückgegebene Objekt ist vom Typ `bool` (ein "Boolean"):

```
In [ ]: type(True)
```

Ein `bool` kann einen von zwei Werten annehmen: `True` oder `False`.

Sie werden von *relationalen Operatoren* zurückgegeben: `<`, `<=`, `>`, `>=`, `==` (Gleichheit), `!=` (Ungleichheit) und können mit den *logischen Operatoren* `and`, `or` und `not` kombiniert werden.

```
In [ ]: 1 <= 2 < 4
```

```
In [ ]: 1 < 2 and 2 < 1
```

```
In [ ]: not(1 < 2)
```

Zeichenketten

Zeichenketten enthalten Text. Sie werden mit einfachen oder doppelten Anführungszeichen erstellt:

```
In [ ]: s1 = "Python"  
s2 = ' ist einfach.'  
s3 = s1 + s2 # Verkettung
```

```
In [ ]: type(s3)
```

Das funktioniert nicht:

```
In [ ]: a = 3 # eine Ganzzahl  
b = "4" # eine Zeichenkette  
# Entkommentieren und ausführen:  
# a + b
```

Wir müssen die Zeichenkette zuerst konvertieren:

```
In [ ]: a + int(b)
```

Wir können auch in die andere Richtung konvertieren:

```
In [ ]: a = 3  
b = str(a)
```

```
In [ ]: type(b)
```

Das ist nützlich für die Ausgabe:

```
In [ ]: height = 1.89  
print("Ich bin " + str(height) + "m gross.")
```

Eine Möglichkeit, eine Zeichenkette zu erhalten, besteht darin, den Benutzer nach einer Eingabe zu fragen:

```
In [ ]: mystr = input("Wie heisst du? ")  
        print(mystr)
```

Übung

Schreiben Sie Code in die Zelle unten, der den Benutzer nach seinem Alter fragt und dann das Alter in Hundejahren ausgibt (d.h. geteilt durch 7).

Beispiel-Eingabe:

```
Wie alt bist du?
```

Wenn der Benutzer `28` eingibt, sollte dies die folgende Ausgabe erzeugen:

```
Dein Alter in Hundejahren ist 4.0.
```

Beachten Sie, dass `input` immer eine Zeichenkette zurückgibt, also müssen Sie sie in `int` (oder `float`) konvertieren, um damit mathematische Operationen durchzuführen.

In []:

Sequenztypen: Container mit ganzzahliger Indizierung

Zeichenketten

Wir haben bereits `Zeichenketten` kennengelernt: Sie enthalten Text und werden mit einfachen oder doppelten Anführungszeichen erstellt:

```
In [ ]: s1 = "Python"; s2 = ' ist einfach.'; s3 = s1 + s2 # Verkettung  
print(s3)
```

Die `len`-Funktion gibt die Länge einer Zeichenkette zurück:

```
In [ ]: len(s3)
```


Wir können Zeichen aus einer Zeichenkette durch *Indizierung* auswählen:

```
In [ ]: print(s3)
```

```
In [ ]: s3[0] # Beachten Sie die nullbasierte Indizierung
```

```
In [ ]: s3[-1] # Negative Indizes zählen von rechts:
```

Wir können auch mehrere Elemente auswählen ("*Slicing*"). Dies funktioniert für alle *Sequenztypen* (Listen, NumPy-Arrays, ...).

```
In [ ]: print(s3)
```

```
In [ ]: s3[0:2] # Elemente 0 und 1; linker Endpunkt ist enthalten, rechter Endpunkt ausgeschlossen.
```

```
In [ ]: s3[0:6:2] # Start:Stop:Schritt
```

```
In [ ]: s3[::-1] # Start und Stop können weggelassen werden; Standard ist 0 und len(str)
```

Übung

Verwenden Sie Slicing, um die Teilzeichenkette `mich` aus der Zeichenkette `s` unten zu extrahieren.

```
In [ ]: s = 'Warum habe ich mich nur hierfür angemeldet?'
```

Hausaufgabe

Anfängerübungen 1-4 von <https://holypython.com/beginner-python-exercises/>

In []:

